



COMANDOS KUBERNETES (K8S) - GUIA COMPLETO



COMANDOS BÁSICOS DO KUBERNETES

Configuração e Informação do Cluster

```
bash
```

```
# Ver configuração do cluster
```

```
kubectl config view
```

```
kubectl config current-context
```

```
kubectl config get-contexts
```

```
# Mudar contexto/namespace
```

```
kubectl config use-context meu-contexto
```

```
kubectl config set-context --current --namespace=meu-namespace
```

```
# Informações do cluster
```

```
kubectl cluster-info
```

```
kubectl version
```

```
kubectl get componentstatuses
```

```
# Ver nodes do cluster
```

```
kubectl get nodes
```

```
kubectl get nodes -o wide
```

```
kubectl describe node <node-name>
```

```
kubectl top node
```

```
# Uso de recursos (requer
```

```
metrics-server)
```



COMANDOS PARA PODS

Gerenciamento de Pods

```
bash
```

```

# Listar pods
kubectl get pods
kubectl get pods -o wide
kubectl get pods --all-namespaces
kubectl get pods -w # Watch mode

# Detalhes do pod
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs -f <pod-name> # Seguir Logs
kubectl logs -f <pod-name> -c <container> # Logs de container específico

# Executar comandos no pod
kubectl exec -it <pod-name> -- /bin/bash
kubectl exec <pod-name> -- ls /app
kubectl exec -it <pod-name> -c <container> -- /bin/bash

# Criar e gerenciar pods
kubectl run nginx --image=nginx # Criar pod imperativo
kubectl delete pod <pod-name>

kubectl delete pods --all # Deletar todos os pods

```



COMANDOS PARA DEPLOYMENTS

Gerenciamento de Deployments

```
bash
```

```

# Listar deployments
kubectl get deployments
kubectl get deploy

# Criar deployment
kubectl create deployment nginx --image=nginx
kubectl apply -f deployment.yaml

# Escalar deployment
kubectl scale deployment nginx --replicas=3
kubectl autoscale deployment nginx --min=2 --max=10 --cpu-percent=80

# Atualizar deployment
kubectl set image deployment/nginx nginx=nginx:1.20
kubectl rollout status deployment/nginx
kubectl rollout history deployment/nginx

```

Rollback

```
kubectl rollout undo deployment/nginx
```

```
kubectl rollout undo deployment/nginx --to-revision=2
```

Deletar deployment

```
kubectl delete deployment nginx
```



COMANDOS PARA SERVICES

Gerenciamento de Services

```
bash
```

Listar services

```
kubectl get services
```

```
kubectl get svc
```

Criar service

```
kubectl expose deployment nginx --port=80 --type=LoadBalancer
```

```
kubectl apply -f service.yaml
```

Port forwarding

```
kubectl port-forward svc/nginx 8080:80
```

```
kubectl port-forward deployment/nginx 8080:80
```

```
kubectl port-forward pod/nginx-pod 8080:80
```

Detalhes do service

```
kubectl describe service nginx
```

```
kubectl delete service nginx
```



COMANDOS PARA CONFIGMAPS E SECRETS

ConfigMaps

```
bash
```

Criar ConfigMap

```
kubectl create configmap my-config --from-literal=key=value
```

```
kubectl create configmap app-config --from-file=config.properties
```

```
kubectl apply -f configmap.yaml
```

Listar ConfigMaps

```
kubectl get configmaps
```

```
kubectl get cm
```

Ver detalhes

```
kubectl describe configmap my-config
```

```
kubectl get configmap my-config -o yaml
```

Secrets

```
bash
```

Criar Secret

```
kubectl create secret generic my-secret --from-literal=password=secret123
```

```
kubectl create secret tls tls-secret --cert=cert.pem --key=key.pem
```

```
kubectl apply -f secret.yaml
```

Listar Secrets

```
kubectl get secrets
```

```
kubectl describe secret my-secret
```

Codificar/decodificar

```
echo -n 'secret' | base64
```

```
echo 'c2VjcmV0' | base64 --decode
```

COMANDOS PARA NAMESPACES

Gerenciamento de Namespaces

```
bash
```

Listar namespaces

```
kubectl get namespaces
```

```
kubectl get ns
```

Criar namespace

```
kubectl create namespace meu-ns
```

```
kubectl apply -f namespace.yaml
```

Trabalhar em namespace específico

```
kubectl get pods -n meu-ns
```

```
kubectl config set-context --current --namespace=meu-ns
```

Deletar namespace

```
kubectl delete namespace meu-ns
```



COMANDOS PARA ESTADO E MONITORAMENTO

Estado e Recursos

```
bash
```

Ver todos os recursos

```
kubectl get all
```

```
kubectl get all -n meu-namespace
```

Uso de recursos

```
kubectl top pods
```

```
kubectl top nodes
```

```
kubectl top pods --containers
```

Events

```
kubectl get events
```

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

```
kubectl get events -w
```



COMANDOS PARA TROUBLESHOOTING

Debug e Diagnóstico

```
bash
```

Ver Logs de múltiplos pods

```
kubectl logs -l app=nginx
```

Executar pod temporário para troubleshooting

```
kubectl run debug-pod --image=busybox --rm -it --restart=Never -- /bin/sh
```

Ver recursos com problemas

```
kubectl get pods --field-selector=status.phase!=Running
```

```
kubectl describe $(kubectl get pods -o name | grep nginx)
```

Ver IP do pod

```
kubectl get pod nginx-pod -o jsonpath='{.status.podIP}'
```

Testar conectividade de rede

```
kubectl run network-test --image=appropriate/curl --rm -it  
--restart=Never -- curl http://service-name
```



MANIFESTOS YAML - EXEMPLOS PRÁTICOS

deployment.yaml

```
yaml
```

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-deployment  
  labels:  
    app: nginx  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: nginx  
  template:  
    metadata:  
      labels:  
        app: nginx  
    spec:  
      containers:  
        - name: nginx
```

```
image: nginx:1.20
ports:
- containerPort: 80
resources:
  requests:
    memory: "64Mi"
    cpu: "250m"
  limits:
    memory: "128Mi"
    cpu: "500m"
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
```

```
    periodSeconds: 10
```

service.yaml

```
yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```

```
  type: LoadBalancer
```

configmap.yaml

```
yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
name: app-config
data:
  database.url: "postgresql://localhost:5432/mydb"
  log.level: "INFO"
  application.properties: |
    server.port=8080
```

```
spring.profiles.active=production
```

secret.yaml

```
yaml
```

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  database-password: c2VjcmV0cGFzc3dvcmQ=
```

```
api-key: YXBpLWtleS1zZWNyZXQ=
```



COMANDOS AVANÇADOS

Jobs e CronJobs

```
bash
```

```
# Gerenciar Jobs
```

```
kubectl get jobs
```

```
kubectl create job my-job --image=busybox -- date
```

```
kubectl delete job my-job
```

```
# Gerenciar CronJobs
```

```
kubectl get cronjobs
```

```
kubectl create cronjob my-cron --image=busybox --schedule="*/1 * * * *
*" -- /bin/sh -c 'date'
```


StatefulSets e Persistent Volumes

```
bash
```

```
# StatefulSets
```

```
kubectl get statefulsets
```

```
kubectl describe statefulset mysql
```

```
# Persistent Volumes
```

```
kubectl get pv
```

```
kubectl get pvc
```

```
kubectl describe pvc my-pvc
```

Ingress

```
bash
```

```
# Gerenciar Ingress
```

```
kubectl get ingress
```

```
kubectl describe ingress my-ingress
```

```
kubectl apply -f ingress.yaml
```



COMANDOS PARA ATUALIZAÇÃO E MANUTENÇÃO

Rollout e Updates

```
bash
```

```
# Estratégias de rollout
```

```
kubectl rollout pause deployment/nginx
```

```
kubectl rollout resume deployment/nginx
```

```
kubectl rollout restart deployment/nginx
```

```
# Canary deployment
kubectl set image deployment/nginx nginx=nginx:1.21
kubectl rollout status deployment/nginx
```

```
# Blue-Green deployment
kubectl apply -f deployment-v2.yaml
```

```
kubectl patch service nginx -p
'{"spec":{"selector":{"version":"v2"}}}'
```

Backup e Restore

```
bash
```

```
# Backup de recursos
kubectl get all --all-namespaces -o yaml > backup.yaml
kubectl get secret regcred -o yaml --export > secret.yaml
```

```
# Aplicar backup
kubectl apply -f backup.yaml
```

```
# Backup com Velero (ferramenta externa)
velero backup create my-backup
```

```
velero restore create --from-backup my-backup
```



COMANDOS PARA DESENVOLVIMENTO

Desenvolvimento Local com Minikube

```
bash
```

```
# Iniciar Minikube
minikube start
minikube start --driver=docker
minikube start --nodes=2
```

```
# Gerenciar Minikube
minikube status
minikube stop
```

```
minikube delete
```

```
# Serviços do Minikube
```

```
minikube dashboard
```

```
minikube service nginx-service --url
```

```
minikube ip
```

Desenvolvimento com Skaffold

```
bash
```

```
# Desenvolvimento com rebuild automático
```

```
skaffold dev
```

```
skaffold run
```

```
skaffold delete
```

```
# Configurar Skaffold
```

```
skaffold init
```



COMANDOS PARA CLOUD

GCP (GKE)

```
bash
```

```
# Gerenciar clusters GKE
```

```
gcloud container clusters list
```

```
gcloud container clusters get-credentials meu-cluster
```

```
# Escalar cluster
```

```
gcloud container clusters resize meu-cluster --num-nodes=3
```

AWS (EKS)

```
bash
```

```
# Gerenciar clusters EKS
```

```
aws eks list-clusters
```

```
aws eks update-kubeconfig --region us-east-1 --name meu-cluster
```

```
# Atualizar EKS
```

```
eksctl get cluster
```

```
eksctl upgrade cluster --name=meu-cluster
```

Azure (AKS)

```
bash
```

```
# Gerenciar clusters AKS
```

```
az aks list
```

```
az aks get-credentials --resource-group meu-rg --name meu-cluster
```

```
# Escalar AKS
```

```
az aks scale --resource-group meu-rg --name meu-cluster --node-count 5
```



COMANDOS PARA MONITORAMENTO E LOGS

Monitoramento com Prometheus

```
bash
```

```
# Ver métricas
```

```
kubectl top pods
```

```
kubectl top nodes
```

```
# Port-forward para Prometheus
```

```
kubectl port-forward -n monitoring svc/prometheus-k8s 9090
```

```
# Consultas Prometheus via kubectl
```

```
kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes | jq .
```

Logs Agregados

```
bash
```

```
# Logs com seletores
```

```
kubectl logs -l app=nginx --tail=50
```

```
kubectl logs --since=1h -l app=nginx
```

```
# Múltiplos containers
```

```
kubectl logs deployment/nginx --all-containers=true
```



SCRIPTS ÚTEIS PARA KUBERNETES

Script de Deploy

```
bash
```

```
#!/bin/bash
```

```
# deploy.sh
```

```
NAMESPACE="production"
```

```
APP_NAME="my-app"
```

```
IMAGE_TAG="v1.2.3"
```

```
echo "Deploying $APP_NAME:$IMAGE_TAG to $NAMESPACE..."
```

```
# Update image
```

```
kubectl set image deployment/$APP_NAME $APP_NAME=my-registry/$APP_NAME:$IMAGE_TAG  
-n $NAMESPACE
```

```
# Wait for rollout
```

```
kubectl rollout status deployment/$APP_NAME -n $NAMESPACE --timeout=300s
```

```
if [ $? -eq 0 ]; then
```

```
    echo "Deployment successful!"
```

```
else
```

```
    echo "Deployment failed! Rolling back..."
```

```
    kubectl rollout undo deployment/$APP_NAME -n $NAMESPACE
```

```
    exit 1
```

```
fi
```

Script de Health Check

```
bash
```

```
#!/bin/bash
```

```
# health-check.sh
```

```
NAMESPACE="default"
```

```
echo "Cluster health check:"
```

```
echo "Nodes:"
```

```
kubectl get nodes
```

```
echo -e "\nPods:"
```

```
kubectl get pods -n $NAMESPACE
```

```
echo -e "\nServices:"
```

```
kubectl get services -n $NAMESPACE
```

```
echo -e "\nDeployments:"
```

```
kubectl get deployments -n $NAMESPACE
```

```
echo -e "\nEvents:"
```

```
kubectl get events -n $NAMESPACE --sort-by=.lastTimestamp | tail -10
```

Script de Cleanup

```
bash
```

```
#!/bin/bash
```

```
# cleanup.sh
```

```
NAMESPACE="development"
```

```
echo "Cleaning up namespace: $NAMESPACE"
```

```
# Delete all resources in namespace
```

```
kubectl delete all --all -n $NAMESPACE
```

```
# Delete configmaps and secrets
```

```
kubectl delete configmaps,secrets --all -n $NAMESPACE
```

```
# Delete PVCs
```

```
kubectl delete pvc --all -n $NAMESPACE
```

```
echo "Cleanup completed!"
```



DICAS E BOAS PRÁTICAS

Comandos Úteis do Dia a Dia

```
bash
```

```
# Alias comuns (adicionar no ~/.bashrc ou ~/.zshrc)
```

```
alias k='kubectl'  
alias kg='kubectl get'  
alias kd='kubectl describe'  
alias kl='kubectl logs'  
alias kaf='kubectl apply -f'  
alias kdf='kubectl delete -f'
```

```
# Auto-completion
```

```
source <(kubectl completion bash)  
echo "source <(kubectl completion bash)" >> ~/.bashrc
```

```
# Contexto e namespace atual no prompt
```

```
export PS1='\u@\h \w (\$(kubectl config current-context):\$(kubectl  
config view --minify -o jsonpath="{.namespace}"))\$ '
```

Debug Avançado

```
bash
```

```
# Ver recursos em formato JSON
```

```
kubectl get pod nginx-pod -o json
```

```
# Custom columns
```

```
kubectl get pods -o  
custom-columns="NAME:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName"
```

```
# Dry-run para testar
```

```
kubectl apply -f deployment.yaml --dry-run=client
```

```
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml
```

Este guia cobre todos os comandos essenciais para trabalhar com

Kubernetes em ambientes de desenvolvimento e produção! 